

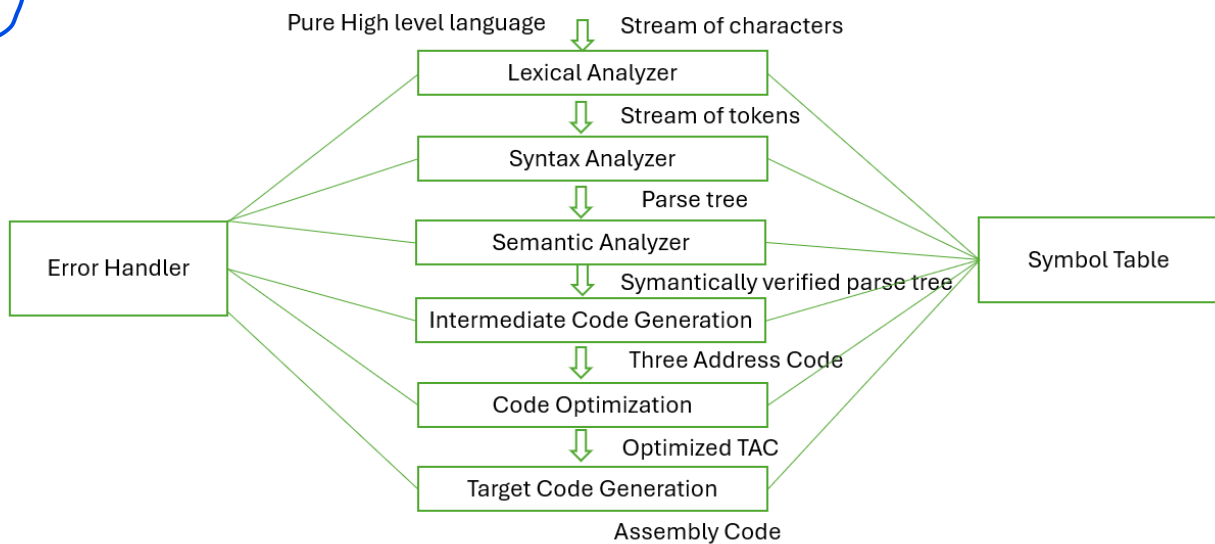
7

Compiler vs Interpreter

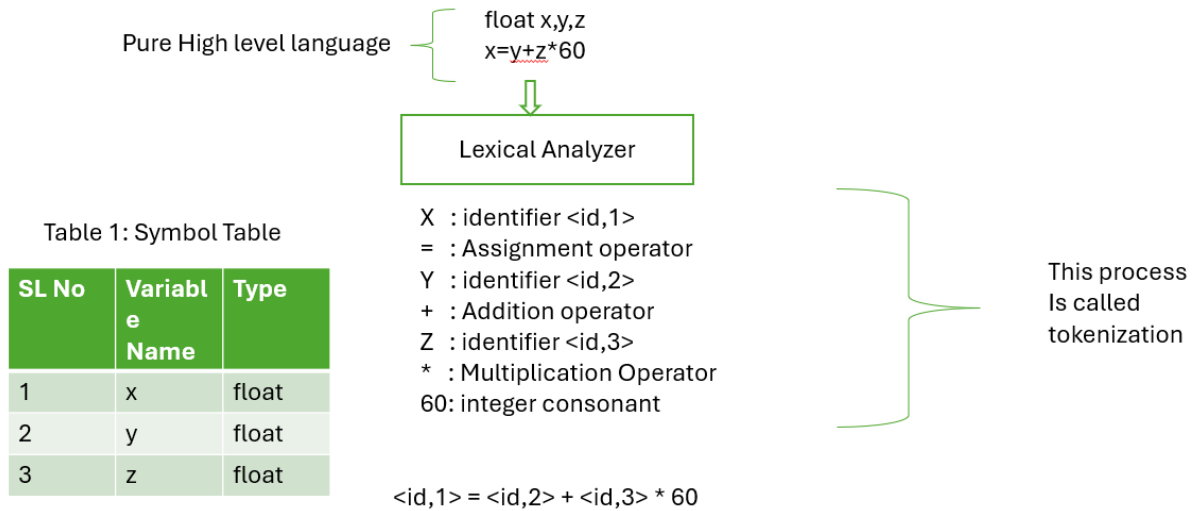
Compiler	Interpreter
Compiler reads the full program.	Interpreter reads line by line.
It generates .exe file.	It doesn't generate .exe file.
Compiled codes run faster than Interpreter. So, it required less time.	Interpreted codes run slower than Compiler. So, it required more time
It does not require source code for later execution.	It requires source code for later execution.
It takes more space.	It takes less space.
Debugging is difficult here.	Debugging is easy here.

8

6 Phases of Compiler



Lexical Analyzer



Syntax Analyzer

$\langle id, 1 \rangle = \langle id, 2 \rangle + \langle id, 3 \rangle * 60$



Syntax Analyzer



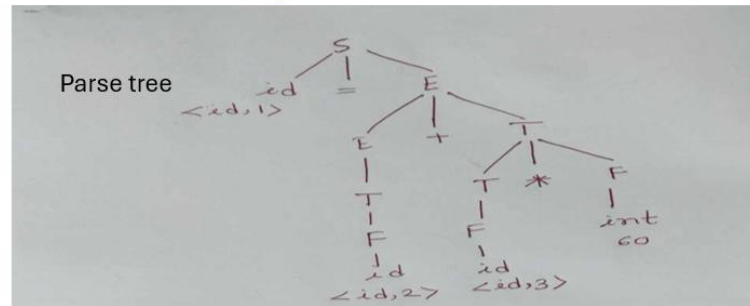
Grammar

$S \rightarrow id = E$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id \mid int$



Semantic Analyzer

Parse tree



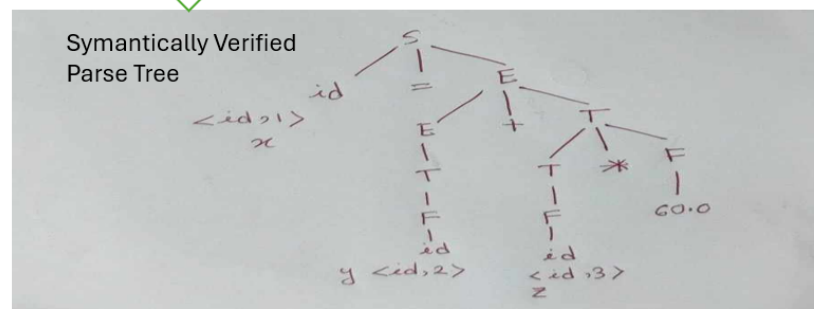
Semantic Analyzer



1. Type Checking
2. Undeclared variable
3. Multiple Declaration

Table 1: Symbol Table

SL No	Variable Name	Type
1	x	float
2	y	float
3	z	float



Intermediate code Generation

- In here the most valuable is three address code.

```
t1 = z*60.0  
t2 = y+t1  
X = t2
```

Code Optimization

- Reduced the no of lines in the code.
- It is optional

```
t1 = z*60.0  
X = y+t1
```

Target Code Generation

- It generates assembly language

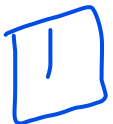
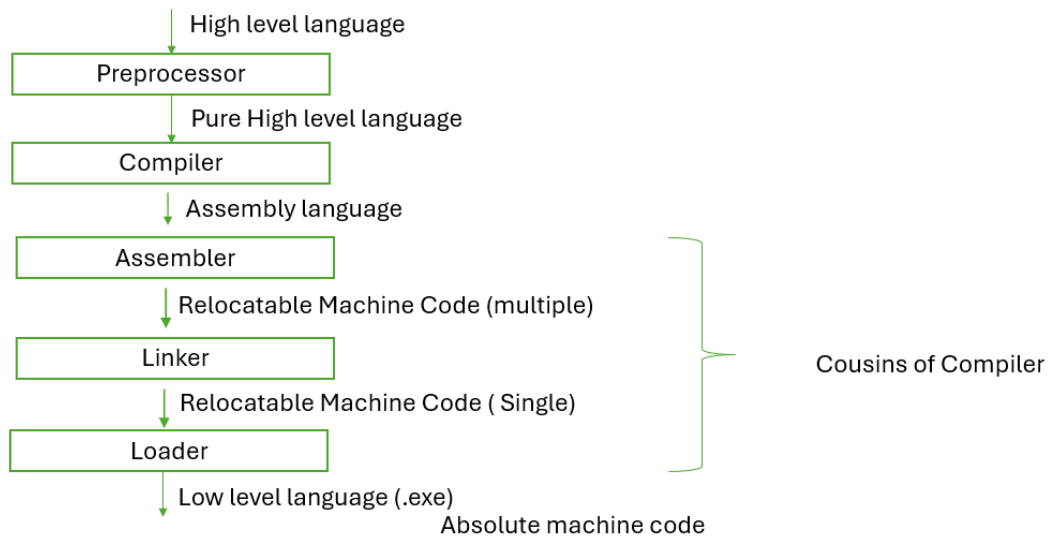
```
MUL    R0, 60.0  
ADD     R1, R0  
STORE  X, R1
```

```
R0 <- z  
R1 <- y
```



The **Logic Processing Unit** is essential for enabling computers to make decisions, control signals, and carry out instructions that rely on logical relationships, not just numeric computation.

LPS (Language Processing System)

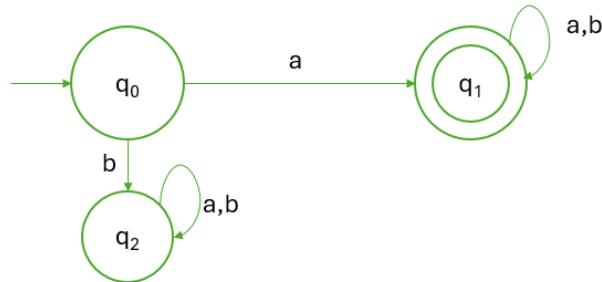


Formal Definition of a DFA

- A DFA can be represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where –
 - Q is a finite set of states.
 - Σ is a finite set of symbols called the alphabet.
 - δ is the transition function where $\delta: Q \times \Sigma \rightarrow Q$
 - q_0 is the initial state from where any input is processed ($q_0 \in Q$).
 - F is a set of final state/states of Q ($F \subseteq Q$).

Example

- **Make a DFA where all strings start with a.**
- You have two alphabets (a,b)
 - { a, aa, aaa, ab, ba }

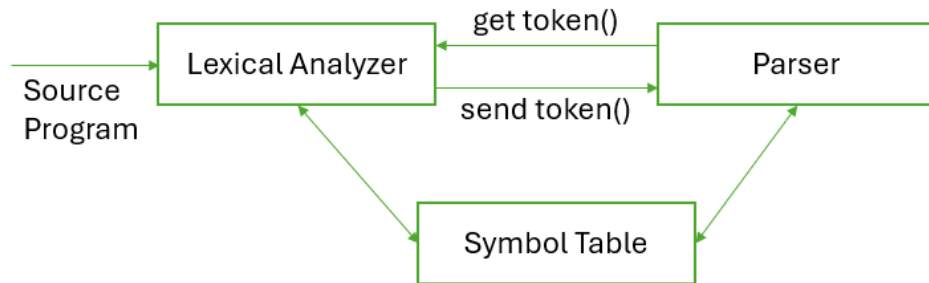


2

Lexical Analyzer

- Lexical analyzer divides the given program into meaningful words which is known as tokens.
- Tokens are normally identifiers, separator, keywords, operators, constant and special symbol.
- The program which did this is called lexer, tokenizer, scanner.
- Lexical analyzer eliminates comments, white space character. (blank, Tab)
- It helps in giving error message such as exceeding length by providing row no and column no.

Lexical Analyzer



Lexical Analyzer

- Lexical analyzer uses DFA to do tokenization.
- While doing tokenization lexical analyzer always gives importance to longest matching.

3

Counting no of tokens (Ex: 6)

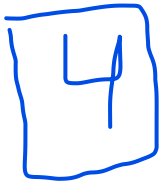
```
1  main() //3
2  { //1
3      char *c="string"; //6
4      float b=100.74; //5
5      char d='e'; //5
6      int f=200; //5
7      in t f=200; //6
8      in /* comment */t f=200; //6
9      ch ar d='e'; //6
10     ch/*comment*/ar d='e';//6
11 } //1
```

Total tokens = 50

Counting no of tokens (Ex: 7)

```
main.c
1  main() //3
2  { //1
3      a=b+++-----+++=; //11
4      x==y /*abcd***/*abcd*/; //9
5      int **p; //5
6      printf("%d %d",a,b); //9
7  } //1
```

Total tokens = 39



Error Recovery

- **Delete:** Unknown characters are deleted. Also known as panic mode recovery. Example:
 - 'Charr' corrected as 'char' deleting r.
- **Insert:** An extra or missing character is inserted to form a meaningful token. Example:
 - 'cha' corrected as 'char' inserting 'r'
- **Transpose:** Based on certain rules we can transpose two characters. Example:
 - 'Whiel' can be corrected to 'while'

Error Recovery

- **Replace:** Based on replacing one character by another. Example:
 - 'chrr' can be corrected as 'char' by replacing 'r' with 'a'.